Supply Chain Security

you ship far more code than you write

Watch the 4-min video

In one line: most of what you ship was written by strangers, so pin and scan dependencies, know what's inside each artifact (SBOM), prove where it came from (signatures and SLSA provenance), verify that at deploy time, and treat the CI pipeline as production.

1 Strangers' code in production

Your own code is a small fraction of what you deploy: open-source packages, base images, build tools and CI plugins make up the rest. Compromise one popular component and you're inside every company that installs it. **SolarWinds** (2020): attackers injected code into the Orion build system, so signed updates carried a backdoor to about 18,000 customers. **xz utils** (2024, CVE-2024-3094): a contributor spent two years earning maintainer trust, then shipped a backdoor targeting SSH; it was caught by chance before reaching most stable distributions.

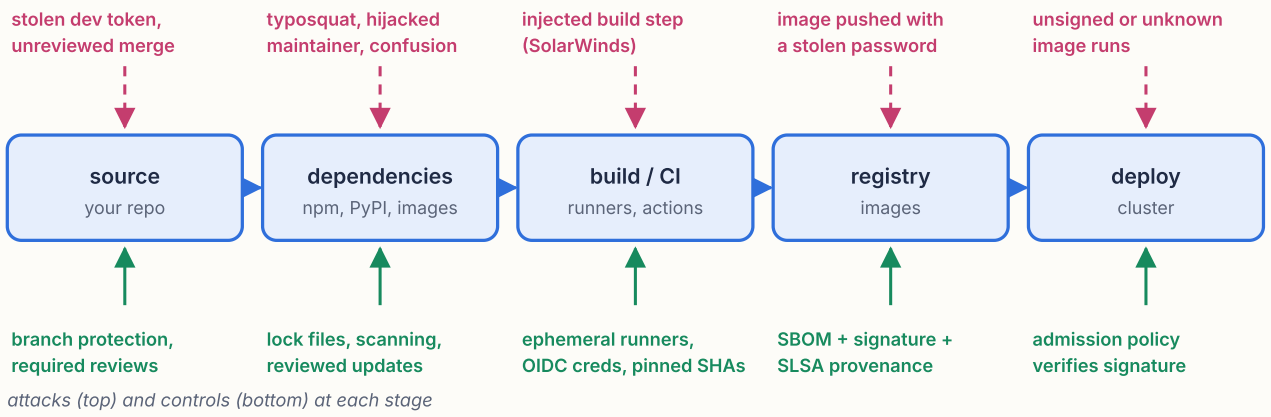


Figure 1. Each stage of the pipeline is an entry point. Controls at every stage make a single stolen credential insufficient.

2 Dependencies

Attack paths: a **hijacked maintainer account** publishes a malicious version; **typosquatting** (`reqeusts` vs `requests`); **dependency confusion**, where a public package with your internal package's name wins the resolver. Defences: **pin exact versions** with lock files (and hashes where supported, e.g. `pip --require-hashes`), scan for known CVEs, let Dependabot or Renovate open update PRs that humans review, use a private registry proxy with scoped names, and keep fewer dependencies.

3 Know what you ship: SBOM

A **software bill of materials** lists every component and version inside an artifact, in a standard format: **SPDX** (ISO/IEC 5962) or **CycloneDX** (OWASP). Generate it in CI for every build (Syft, Trivy) and attach it to the image. When the next Log4Shell lands, a query over SBOMs tells you in minutes which services include `log4j-core 2.14`, instead of days of grepping.

4 Prove where it came from

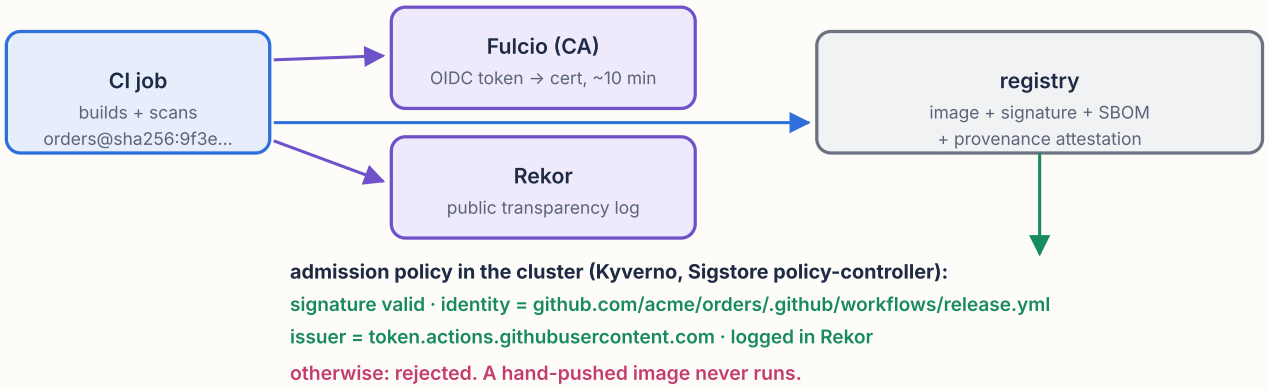


Figure 2. Keyless signing with Sigstore: the CI job's OIDC identity gets a short-lived certificate, and the signature is recorded in a public log.

Sign images in CI and verify before deploy, so the cluster refuses anything unsigned or signed by the wrong identity. **SLSA** (slsa.dev) adds **provenance**: a signed record of which source commit, build definition and builder produced the artifact. SLSA v1.0's build track runs from L1 (provenance exists) to L3 (hardened, isolated builder). A stolen registry password alone can no longer get code into production.

5 Harden the pipeline

- **Short-lived credentials**: OIDC federation from CI to the cloud instead of stored keys (lesson 48).
- **Pin third-party actions to a full commit SHA**, not a tag: in March 2025 the `tj-actions/changed-files` tags were repointed to code that dumped CI secrets.
- **Required reviews** and branch protection on main; no direct pushes, including for admins.
- **Ephemeral, isolated runners**, so one build can't poison the next; least-privilege tokens per job.

WORKED EXAMPLE: SIGNING IMAGES WITH COSIGN

After building and scanning `ghcr.io/acme/orders`, the release workflow runs `cosign sign ghcr.io/acme/orders@sha256:9f3e…`. Keyless mode exchanges the job's GitHub OIDC token for a Fulcio certificate valid for minutes and records the signature in Rekor; `cosign attest` attaches the CycloneDX SBOM and SLSA provenance. In the cluster, an admission policy verifies the signature, the expected workflow identity and issuer. An image pushed by hand, or by anyone else, is rejected. Sign and deploy by **digest**, not by mutable tag.

COMMON MISTAKES

- Floating versions (`^1.2`, `latest`) so builds change silently.
- Auto-merging every dependency update without review or tests.
- Long-lived cloud keys stored as CI secrets, readable by any workflow.
- Signing images but never verifying signatures at deploy time.

6 Key terms

Supply chain attack: compromise via a dependency, tool or pipeline

SBOM: list of all components and versions in an artifact

Typosquatting: malicious package with a look-alike name

Provenance: signed record of how and from what an artifact was built

SLSA: framework of build-integrity levels

Sigstore / cosign: keyless signing with short-lived certs + transparency log

7 Interview questions

Q1. What does an SBOM give you? BASIC

A list of every component and version in an artifact, so when a new CVE lands you know within minutes which services are affected.

Q2. Why pin dependencies with a lock file? BASIC

Builds become reproducible and nothing changes silently; a malicious new version can't slip in unless someone updates and reviews the lock file.

Q3. How do you secure your software supply chain? SCENARIO

Pin and scan dependencies, review automated updates, generate an SBOM per build, sign images in CI and verify at deploy, record SLSA provenance, and harden CI with short-lived credentials, pinned actions, required reviews and ephemeral runners.

Q4. A critical CVE is announced in a popular library. What do you do first? SCENARIO

Query SBOMs to find affected services and versions, prioritise internet-facing ones, patch or mitigate (WAF rule, config), rebuild and redeploy signed images, and verify.

Q5. Why pin a GitHub Action to a commit SHA rather than a version tag? TRICKY

Tags are mutable: whoever controls the action's repo can move v4 to malicious code. A full SHA is immutable, so you run exactly what you reviewed.

Q6. If the source code is reviewed, why isn't that enough? TRICKY

The build can inject code the source never contained (SolarWinds), and dependencies change underneath you. Provenance and signatures prove the artifact came from that reviewed source through a trusted builder.

Next: SD-N-001 · Observability Fundamentals backendroadmap.com